PRÁCTICA OPENCODE UT2. ANALIZAR UNA ARQUITECTURA CON UN AGENTE

Antes de empezar

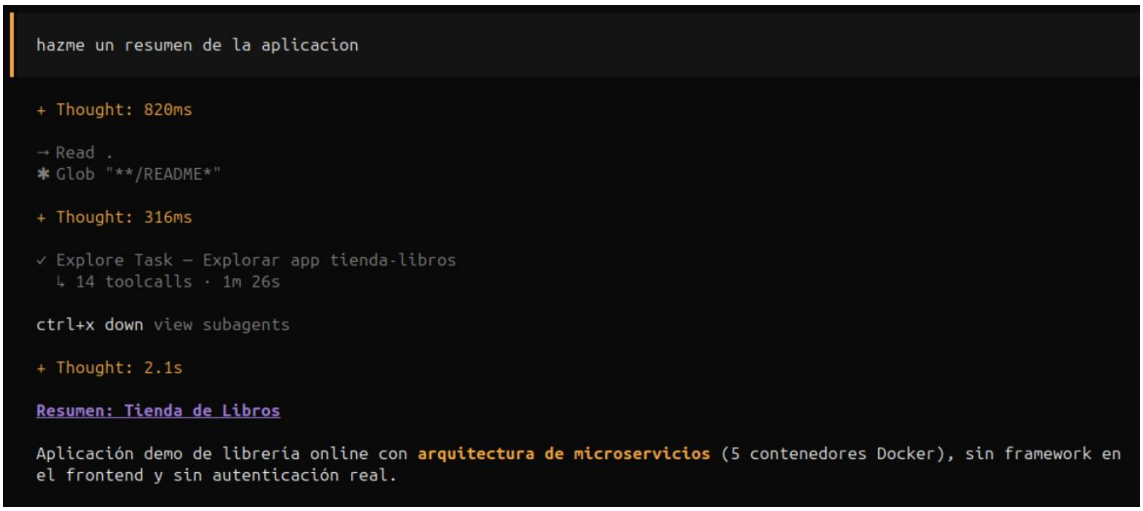
opencode instalado (ver la **guía de instalación**) y el proyecto en un repositorio Git, que es tu botón de deshacer. Comprueba con `/models` qué modelo tienes activo: con los modelos gratuitos tendrás que ser más explícito en los prompts y cometerán más errores, lo que viene bien para lo que se pide aquí. Necesitas además la aplicación `tienda-libros` de la carpeta `recursos`, copiada a tu carpeta de trabajo, y un visor de Markdown que dibuje diagramas Mermaid (Visual Studio Code con la extensión «Markdown Preview Mermaid Support», o el propio GitHub). No hace falta levantar la aplicación.

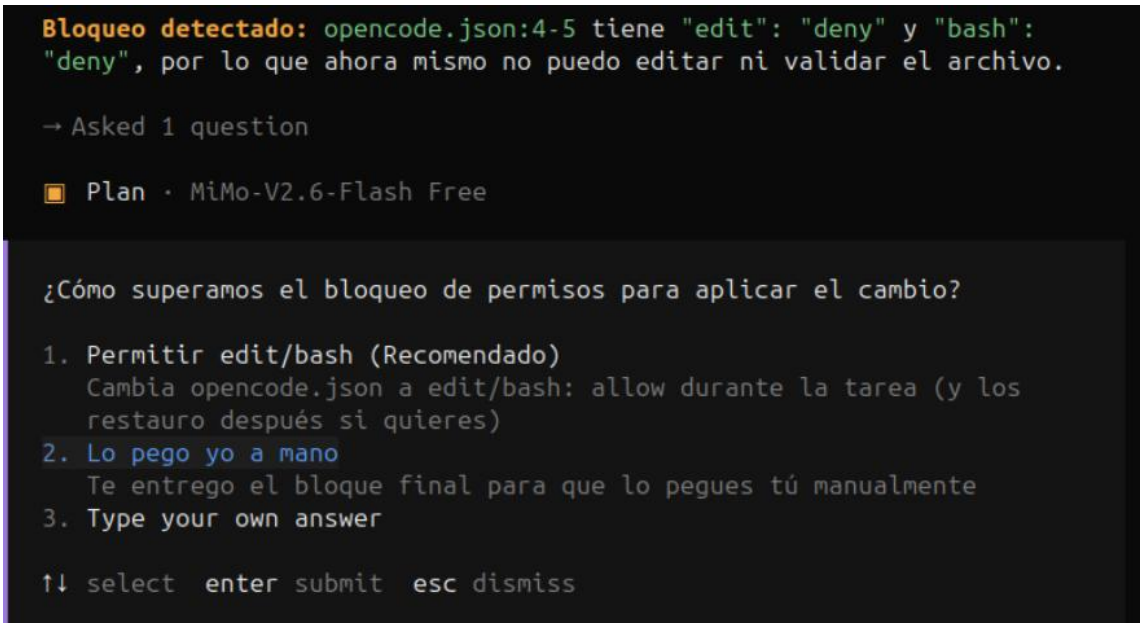
- Trabaja en **modo Plan** (tecla Tab) hasta que el plan te convenza; solo entonces pasa a **Build**.
- Revisa cada cambio antes de aceptarlo (`git diff`). Si no sabes explicar una línea, no la aceptes.
- **Verifica ejecutando**: la evidencia es la salida del comando, no la respuesta del agente.
- Anota cada prompt en `prompts.md`: qué pediste, qué hizo y qué corregiste.
- Nada de contraseñas ni datos personales en el chat: los modelos gratuitos son de prueba y no admiten información confidencial.
- Si el modelo se atasca, da vueltas o va muy lento, divide la petición en pasos o cambia de modelo con `/models`.

Ejercicios

Todo el trabajo se recoge en un documento `docs/arquitectura.md` dentro del proyecto, que escribes **tú** con la ayuda del agente.

1) **Agente en solo lectura.** Crea `opencode.json` con `"edit": "deny"`, `"bash": "ask"` y `"share": "disabled"`, arranca opencode en **modo Plan** y pide un resumen de la aplicación. Después pídele expresamente que modifique `docker-compose.yml`: captura el rechazo. Explica por qué en una fase de análisis conviene un agente que no pueda escribir. (1p)





He cambiado los permisos del bash ya que no reflejaba la denegación. Estaba preguntándome todo el rato si podía ejecutar ciertos comandos.

En una fase de análisis conviene utilizar un agente que no pueda escribir porque permite inspeccionar la aplicación sin introducir cambios accidentales. Así se evita alterar archivos de configuración, romper dependencias o modificar servicios antes de comprender cómo funciona el proyecto.

2) **Inventario de componentes.** Pide al agente la lista de servicios y rellena esta tabla comprobando **cada celda** en los ficheros: servicio, imagen o construcción, puerto publicado al exterior, puerto interno, volúmenes, de qué servicios depende y qué hace. Indica cuántos servicios son accesibles desde fuera del equipo y por qué. (1,5p)

proxy

Campo	Valor
Fichero	docker-compose.yml:2
Servicio	proxy
Imagen	nginx:1.27-alpine (sin build)
Construcción	— (imagen oficial)
Puerto externo	8080 → 80
Puerto interno	80
Volúmenes	./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro, ./web:/usr/share/nginx/html:ro
Depende de	api
Qué hace	Servidor web/front: sirve la SPA estática de web/index.html con fallback SPA (try_files ... /index.html) y hace proxy inverso de /api/ hacia http://api:3000/ (nginx/default.conf:12).

api

Campo	Valor
Fichero	docker-compose.yml:13
Servicio	api
Imagen	node:20-alpine (base del build)
Construcción	build: ./api → api/Dockerfile: npm install --omit=dev, copia src/, EXPOSE 3000, CMD node src/server.js
Puerto externo	— (no publicado)
Puerto interno	3000 (PORT: 3000)
Volúmenes	— (solo env: DATABASE_URL, REDIS_URL, JWT_SECRET)
Depende de	db (condition: service_healthy), cache (condition: service_started)
Qué hace	API REST Express. Rutas: GET /salud (healthcheck), GET /libros (consulta PostgreSQL con caché Redis en clave libros, TTL 60 s), POST /pedidos (inserta pedido y hace lPush del id en la cola Redis cola:facturas).

worker

Campo	Valor
Fichero	docker-compose.yml:28
Servicio	worker
Imagen	node:20-alpine (base del build)
Construcción	build: ./worker → worker/Dockerfile: npm install --omit=dev, copia worker.js, CMD node worker.js
Puerto externo	—
Puerto interno	— (sin EXPOSE, proceso sin red)
Volúmenes	— (solo env REDIS_URL)
Depende de	cache
Qué hace	Consumidor de la cola: bucle brPop('cola:facturas', 0) que retira ids de pedido y "genera la factura" (worker/worker.js:9-10).

db

Campo	Valor
Fichero	docker-compose.yml:36
Servicio	db
Imagen	postgres:16-alpine (sin build)
Construcción	—
Puerto externo	— (no publicado; solo accesible por red interna)
Puerto interno	5432
Volúmenes	datos-db:/var/lib/postgresql/data, ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
Depende de	—
Qué hace	PostgreSQL 16. DB tienda, usuario tienda, password de POSTGRES_PASSWORD. init.sql crea tablas libros y pedidos y las siembra con 3 libros. Healthcheck pg_isready -U tienda (10 s/5 s/5 reintentos).

cache

Campo	Valor
Fichero	docker-compose.yml:52
Servicio	cache
Imagen	redis:7-alpine (sin build)
Construcción	—
Puerto externo	—
Puerto interno	6379
Volúmenes	datos-cache:/data
Depende de	—
Qué hace	Redis 7 con persistencia RDB (--save 60 1). Doble uso: caché de GET /libros y cola de trabajos (cola:facturas) consumida por worker.

adminer

Campo	Valor
Fichero	docker-compose.yml:59
Servicio	adminer
Imagen	adminer:4 (sin build)
Construcción	—
Puerto externo	8081 → 8080
Puerto interno	8080
Volúmenes	— (env ADMINER_DEFAULT_SERVER: db)
Depende de	db (condition: service_healthy)
Qué hace	GUI web para administrar PostgreSQL, preconfigurada para apuntar al host db.

2 servicios son accesibles desde fuera del equipo (desde el host o la red externa):

Servicio	Publicación	Por qué
proxy	8080:80 (docker-compose.yml:5)	Tiene clave ports, así que Docker publica el puerto 8080 del host contra el 80 del contenedor. Es la puerta de entrada: web estática + proxy de /api/.
adminer	8081:8080 (docker-compose.yml:62)	También tiene ports; publica el 8081 del host contra el 8080 del contenedor. Permite administrar la BD sin entrar al contenedor.

3) **Recursos de la aplicación.** Clasifica todos los recursos del proyecto (contenido estático, código de servidor, datos y esquema, dependencias, configuración, secretos, registros) indicando en qué fichero o servicio vive cada uno y cuáles **no deben subirse al repositorio**. Comprueba si el proyecto tiene `.gitignore` y propón uno. (1,5p)

1. Contenido estático

Recurso	Fichero / Servicio	¿Subir al repo?
Frontend HTML + JS inline que consume /api/libros	web/index.html	Sí (código fuente)
Ficheros servidos en runtime	montado en proxy → /usr/share/nginx/html	N/A (copia de web/)

2. Código de servidor

Recurso	Fichero / Servicio	¿Subir?
API Express (rutas /salud, /libros, /pedidos)	api/src/server.js	Sí
Worker de facturas (BRPOP sobre cola:facturas)	worker/worker.js	Sí
Build de imágenes	api/Dockerfile, worker/Dockerfile	Sí

3. Datos y esquema

Recurso	Fichero / Servicio	¿Subir?
DDL: tablas libros, pedidos	db/init.sql	Sí (esquema)
Seed de 3 libros de ejemplo	db/init.sql:15-18	Sí (datos ficticios)
Datos reales (pedidos, ventas)	volumen Docker datos-db (db)	No (fuera del repo; si se bind-muntease a ./datos, ignorarlo)
Persistencia Redis / RDB	volumen Docker datos-cache (cache)	No
Caché libros (TTL 60 s)	servicio cache	No (derivado)
Cola cola:facturas	servicio cache	No (efímero)

4. Dependencias

Recurso	Fichero / Servicio	¿Subir?
express, pg, redis, jsonwebtoken	api/package.json	Sí
redis	worker/package.json	Sí
package-lock.json	**no existe** en api/ ni worker/	Sí, y debe generarse (falta → builds no reproducibles)
node_modules/	generados en la imagen	No (nunca)
Imágenes base node:20-alpine, nginx:1.27-alpine, postgres:16-alpine, redis:7-alpine, adminer:4	docker-compose.yml	Sí (pinned por tag)

5. Configuración

Recurso	Fichero / Servicio	¿Subir?
Orquestación de los 6 servicios, puertos, volúmenes, depends_on	docker-compose.yml	Sí
Virtual host nginx (SPA + proxy /api/)	nginx/default.conf	Sí
Variables de entorno por servicio (PORT, DATABASE_URL, REDIS_URL, JWT_SECRET, ADMINER_DEFAULT_SERVER)	docker-compose.yml	Sí (sin valores secretos)
Config del propio opencode	opencode.json	Sí (no es secreto)
.dockerignore	**no existe** en api/ ni worker/	Recomendable crearlo

6. Secretos

Recurso	Fichero / Servicio	¿Subir?
POSTGRES_PASSWORD real	.env (**no existe aún**; solo la plantilla .env.example)	No, nunca
JWT_SECRET real	.env	No, nunca
Plantilla con valores de ejemplo (cambiame)	.env.example	Sí (sin valores reales)
Password de BD en runtime	variable POSTGRES_PASSWORD inyectada en db y api	N/A (solo en el entorno del contenedor)

7. Registros (logs)

Recurso	Fichero / Servicio	¿Subir?
API escuchando	stdout del contenedor api → docker logs	No
Worker de facturas en marcha / Generando la factura del pedido N	stdout del contenedor worker	No
Logs de PostgreSQL y Redis	stdout de db / cache (no van al volumen)	No
Logs de Adminer / nginx	stdout de adminer / proxy	No
Ficheros *.log sueltos	—	No (añadir a ignore por si acaso)

No existe .gitignore (ni .dockerignore) en el proyecto. Inventario completo de ficheros — 13, todos rastreables salvo los ignorables:

```
api/src/server.js        api/Dockerfile        api/package.json
worker/worker.js         worker/Dockerfile     worker/package.json
nginx/default.conf       docker-compose.yml    db/init.sql
web/index.html           .env.example          opencode.json
```

Además: **no existe .git** — el directorio aún no es un repositorio git, así que no hay historial al que filtrar nada. Buenas noticias: todavía no se ha filtrado ningún secreto.

Propuesta: .gitignore en la raíz

```
# Secretos y variables de entorno
.env
.env.local
.env.*.local
!.env.example

# Dependencias (nunca se suben)
node_modules/
vendor/

# Datos y persistencia local
datos-db/
datos-cache/
data/
*.rdb
dump.rdb

# Registros
*.log
logs/
npm-debug.log*
yarn-error.log*

# Artefactos de build y cobertura
dist/
build/
coverage/
*.tsbuildinfo

# Editores y sistema operativo
.idea/
.vscode/
.DS_Store
Thumbs.db
```

4) **Recorrido de una petición.** Pide al agente dos diagramas **Mermaid**: el de `GET /api/libros` (incluyendo la caché) y el de `POST /api/pedidos` (incluyendo la cola de facturas y el worker). Comprueba que el diagrama coincide con el código de `api/src/server.js` y `worker/worker.js`, corrígelo y verifica que se dibuja en el visor. (1,5p)

1. GET /api/libros (con caché de Redis)

```
sequenceDiagram
    autonumber
    actor Cliente as Cliente
    participant Proxy as proxy nginx
    participant API as api Express
    participant Cache as cache Redis
    participant DB as db Postgres

    Cliente->>Proxy: GET /api/libros
    Proxy->>Proxy: location /api/ hace match
    Proxy->>API: GET /libros (el proxy_pass elimina el prefijo /api)
    API->>Cache: GET libros
    alt Cache HIT - clave libros vigente
        Cache-->>API: JSON con la lista de libros
        API-->>Proxy: 200 application/json desde cache
    else Cache MISS - clave ausente o expirada
        Cache-->>API: null
        API->>DB: SELECT id, titulo, autor, precio FROM libros ORDER BY id
        DB-->>API: rows - 3 libros del seed
        API->>Cache: SET libros con JSON y EX 60
        API-->>Proxy: 200 application/json con rows
    end
    Proxy-->>Cliente: 200 application/json

    Note over API,Cache: TTL de 60s - al expirar la siguiente peticion vuelve a Postgres<br/>cache.
connect solo registra el fallo de Redis, no hay try/catch en el handler<br/>si Redis cae el cache.
get lanza y la peticion termina en 500
```

2. POST /api/pedidos (cola de facturas + worker)

```
sequenceDiagram
    autonumber
    actor Cliente as Cliente
    participant Proxy as proxy nginx
    participant API as api Express
    participant DB as db Postgres
    participant Cache as cache Redis
    participant Worker as worker Node

    Cliente->>Proxy: POST /api/pedidos con libroId y unidades
    Proxy->>API: POST /pedidos
    alt Faltan libroId o unidades
        API-->>Proxy: 400 con error Faltan datos
        Proxy-->>Cliente: 400
    else Datos validos
        API->>DB: INSERT INTO pedidos libro_id y unidades RETURNING id
        DB-->>API: id generado - por ejemplo 1
        API->>Cache: LPUSH cola:facturas con el id como texto
        API-->>Proxy: 201 con el id del pedido
        Proxy-->>Cliente: 201 con el id del pedido

        Note over Worker,Cache: Contenedor independiente - bucle infinito
        Worker->>Cache: BRPOP cola:facturas con timeout 0 - bloqueante
        Cache-->>Worker: id del pedido
        Worker->>Worker: console.log Generando la factura del pedido
    end
```

El error: no existe el "500"

Mis notas decian que un fallo de Redis "termina en 500". **Es incorrecto**:

- `express ^4.19.2` (`api/package.json:10`) **no** enruta rechazos de promesas a su middleware de error: un `await` que revienta en un handler `async` no produce una respuesta, produce una *unhandled rejection*.
- Node 20 (`api/Dockerfile:1`) tiene `--unhandled-rejections=throw` por defecto → el proceso **aborta**.
- Con `restart: unless-stopped` (`docker-compose.yml:26`) el contenedor de la API **se reinicia en bucle** y nginx devuelve **502** al cliente, no 500.

Ramas de error que faltaban

1. **FK inválida**: `pedidos.libro_id REFERENCES libros(id)` (`init.sql:10`). Un `libroId` inexistente hace fallar el `INSERT` → misma *unhandled rejection* → el proceso muere. El diagrama no lo contemplaba y **no hay ningún 4xx/5xx limpio** que lo exprese.
2. **Worker sin Redis al arrancar**: `await cache.connect()` (`worker.js:6`) no está envuelto en `catch` y `main()` (`worker.js:14`) se llama sin `.catch()` → rejection no capturada → el worker muere y se reinicia (`docker-compose.yml:34`).

5) **Modelo de arquitectura.** Pregunta al agente qué modelo de arquitectura sigue la aplicación y por qué. Contrasta su respuesta con la teoría de la unidad y con los ficheros: justifica el modelo con **evidencias concretas** (número de despliegues independientes, quién habla con la base de datos, cómo se comunican los servicios) y señala qué parte de la respuesta del agente es una generalidad de manual que no se sostiene aquí. Añade ventajas e inconvenientes de este diseño para una tienda pequeña. (1,5p)

He re-leído todos los ficheros del proyecto para fundamentar el análisis. Veredicto: **arquitectura de microservicios con comunicación híbrida (REST síncrono + cola asíncrona), estilo transaction-script dentro de cada servicio, cache-aside en lectura y work queue en el proceso de facturación.**

1. Modelo global: microservicios en monorepo, despliegue en un solo host

Evidencia	Lectura
6 servicios con ciclo de vida propio en `docker-compose.yml`	Desacoplamiento de despliegue: cada contenedor se reinicia, escala y falla por separado
Red `bridge` interna; solo `proxy` y `adminer` publican puertos	Perímetro de seguridad: la topología interna queda oculta
`build: ./api` / `build: ./worker` con Dockerfiles propios	Cada servicio tiene su propio ciclo de build e imagen
`restart: unless-stopped` en todos	Docker Compose como orquestador mínimo (no Kubernetes, ni service mesh)

ppy Disk

Por qué: es un despliegue de un solo host con Compose, pensado paraaylor la topología con un solo fichero declarativo en vez de un clúster. El coste es que no hay balanceo, ni réplicas, ni rolling updates; `depends_on` solo ordena el arranque, no coordina ejecución.

Por qué: es un despliegue de un solo host con Compose, pensado paraaylor la topología con un solo fichero declarativo en vez de un clúster. El coste es que no hay balanceo, ni réplicas, ni rolling updates; `depends_on` solo ordena el arranque, no coordina ejecución.

2. Patrones por capa

Capa	Patrón	Dónde vive	Por qué
Entrada	**Reverse proxy / puerta única**	`nginx/default.conf:8-16`	Un solo origen público (8080). Sirve estáticos y enruta `/api/`; permite añadir TLS, rate limiting o WAF sin tocar la app
Frontend	**Estático sin build**	`web/index.html`	Un HTML con `fetch('/api/libros')` en línea. Cero toolchain: el navegador es el "compilador"
API	**Transaction Script** (rutas = unidades de trabajo)	`api/src/server.js:12-31`	Sin capas *service* / *repository* / *DAO*: la ruta consulta, valida y responde. proportionality con un dominio de 2 tablas
API	**Repository implícito**: el `Pool` de `pg` es la única puerta a datos	`api/src/server.js:8`	La API es la **única** que toca Postgres; el worker no conoce el esquema
Lectura	**Cache-Aside**	`api/src/server.js:15-19`	Lee de Redis, si falla va a Postgres y rellena con `EX 60`. TTL corto porque el catálogo cambia poco y así

Escritura	**Dual-write (⚠️)**	api/src/server.js:25-30	INSERT en Postgres + LPUSH en Redis son dos escrituras sin transacción común
Asíncrono	**Work queue / Productor-Consumer**	api/src/server.js:29 (productor) + worker/worker.js:9 (consumidor)	LPUSH + BRPOP = FIFO estricto. Desacopla el pedido (rápido, transaccional) de la factura (lenta, reintentable). La API no bloquea al usuario esperando la factura
Procesos	**Backend asíncrono para tareas en segundo plano**	worker/	Es el patrón canónico para no meter trabajo pesado en el ciclo petición/respuesta
Datos	**Base de datos compartida, esquema único**	db/init.sql (2 tablas)	Sencillo y sin duplicar datos. El coste es acoplamiento: cualquier cambio en pedidos afecta a quien lo lea
Config	**12-factor (parcial)**: config por env, un proceso por contenedor, logs a stdout	docker-compose.yml:15-20	Despliegue reproducible: la imagen es inmutable y todo el entorno es explícito
Topología	**IaC declarativa**	docker-compose.yml entero	El fichero *es* la documentación ejecutable del sistema

Se trata de una aplicación web en capas (3 capas, cliente-servidor) con un backend monolítico y una cola de tareas en segundo plano. Se despliega en varios contenedores, pero no es una arquitectura de microservicios.

Evidencias en los ficheros

Despliegues independientes: hay 5 contenedores, pero solo 2 aplicaciones propias (api y worker, con su Dockerfile), los otros 3 (nginx, postgres, redis) son infraestructura estándar. Todo el dominio de negocio (catálogo y pedidos) está en un único código, server.js (33 líneas), y la "facturación" es un bucle de 14 líneas. Un solo docker-compose.yml lo levanta todo junto, en el mismo host y con el mismo ciclo de versiones. En microservicios habría un servicio por capacidad de negocio (catálogo, pedidos, facturación) desplegable y escalable por separado.

Quien habla con la base de datos: Solo la API

Como se comunican:

1.Del Navegador a la API mediante peticiones HTTP.

2. De la API a Postgres y a Redis.

3. De la API al worker mediante una lista de Redis.

La generalidad principal que dijo es que es una **arquitectura de microservicios**.

Ventajas e inconvenientes para una tienda pequeña.

Ventajas

- Sencillo de operar. Haces un docker compose up y ya lo tienes todo listo.
- Barato y fácil de entender
- Rápido para el cliente

Inconvenientes

- Punto único de fallo. Si cae el host, cae todo
- Cola poco fiable.

- Sin HTTPS ni auth
- Falta de robustez

6) **Ficha de requerimientos de implantación.** Con ayuda del agente, redacta la ficha para poner esta aplicación en producción: hardware o instancia, software y versiones, red (dominio, DNS, puertos, cortafuegos), seguridad y secretos, entornos, datos y copias de seguridad, monitorización y registros, fases del despliegue y plan de vuelta atrás. La ficha debe distinguir con claridad **lo que la aplicación ya trae** de **lo que hay que añadir** para producción. (2p)

Resumen: ya trae vs. hay que añadir

Área	✅ Ya lo trae	➕ Imprescindible añadir
Hardware	Despliegue en contenedores, sin dependencias externas	VPS 2 vCPU/4 GB/SSD, swap, límites de recursos
Software	Compose con 6 servicios, versiones fijadas (Node 20, PG 16, Redis 7, nginx 1.27)	Lockfiles + npm ci, digest de imágenes, certbot, migraciones
Red	Solo nginx y Adminer publican puertos; el resto, red interna	Dominio + DNS A, puertos 80/443, ufw, TLS, cerrar 8080/8081
Seguridad	Secretos en .env, NODE_ENV=production, SQL parametrizado	Secretos fuertes y con permisos 600, quitar Adminer, usuario no-root, .gitignore, límites de body
Entornos	Uno con NODE_ENV=production y plantilla .env.example	docker-compose.prod.yml, .env por entorno, etiquetas git/imágenes
Datos	Volúmenes persistentes, healthcheck, init.sql	Migraciones, pg_dump diario fuera del servidor, AOF/colas, pruebas de restore
Monitorización	/salud y logs a consola	Sondeo + alertas, rotación de logs, alertas de disco/certificado
Despliegue	restart: unless-stopped, arranque ordenado por depends_on	Fases 0-5, tags de imagen, smoke tests
Vuelta atrás	Volumes e imágenes conservan el estado	Plan A/B/C con pasos, disparadores, RTO/RPO

7) **Caza de afirmaciones falsas.** A lo largo de la práctica, el agente dirá cosas que no son ciertas para esta aplicación. Recoge al menos **tres**, cada una con la frase del agente, el fichero y la línea que la desmienten y la afirmación correcta. (1p)

“Es una arquitectura de microservicios”	docker-compose.yml. 1. 14 y 29 (solo 2 servicios con build) y 18 y 31 (DATABASE_URL solo en la API)	Es una arquitectura en capas con backend monolítico y un worker , hay una sola base de datos y solo la API accede a ella
La API está expuesta en el puerto 3000	docker-compose.yml 1. 4-5 (solo proxy tiene ports), api/Dockerfile linea 6 (EXPOSE no publica)	Desde fuera solo se llega al 8080 del proxy, la API va por /api/
El worker genera facturas	worker.js 1. 10 (solo console.log), compose linea 30-31 (sin DATABASE_URL)	El worker solo escribe una línea en el log, no accede a la base de datos

Reto (ampliación)

Repite el ejercicio 2 con dos prompts distintos: uno vago («explícame la arquitectura») y otro preciso, citando ficheros con `@docker-compose.yml` `@api/src/server.js` y diciendo qué formato quieres. Compara las dos respuestas y explica qué cambia. Publica después `docs/arquitectura.md` en el repositorio de la unidad para que se vea el diagrama renderizado.

```
Explícame la arquitectura.

+ Thought: 1.2s

Arquitectura de la Tienda de Libros

                                  Host (VPS / Docker Compose)
 Navegador ──HTTP:8080──▶ proxy (nginx:1.27) ──▶ /usr/share/nginx/html  (web/estática: index.html)
                          └─ /api/* ──▶ api (Node 20 + Express, puerto 3000)
                                          ▼                ▼                ▼
                                  db (PostgreSQL 16)  cache (Redis 7)  worker (Node 20)
                                  volumen datos-db    volumen datos-cache  (solo Redis)
                                  adminer:4 (puerto 8081) ──▶ db   [herramienta de desarrollo]

Servicios (6 contenedores en docker-compose.yml)

| Servicio | Imagen | Rol | Puertos |
| proxy | nginx:1.27-alpine | Punto de entrada único: sirve la web estática y hace de reverse proxy de la API | 8080→80 público |
| api | build ./api (node:20-alpine) | API REST: GET /salud, GET /libros, POST /pedidos | internos (3000) |
| worker | build ./worker (node:20-alpine) | Procesa en bucle la cola cola:facturas de Redis ("generar facturas") | ninguno |
| db | postgres:16-alpine | Almacén persistente: tablas libros y pedidos (semilla en db/ | internos (5432) |
| cache | redis:7-alpine | Doble uso: caché de GET /libros (TTL 60 s) y cola de mensajes (lPush/brPop) | internos (6379) |
| adminer | adminer:4 | UI web para consultar la BD (solo desarrollo) | 8081→8080 público |

Flujo de una petición

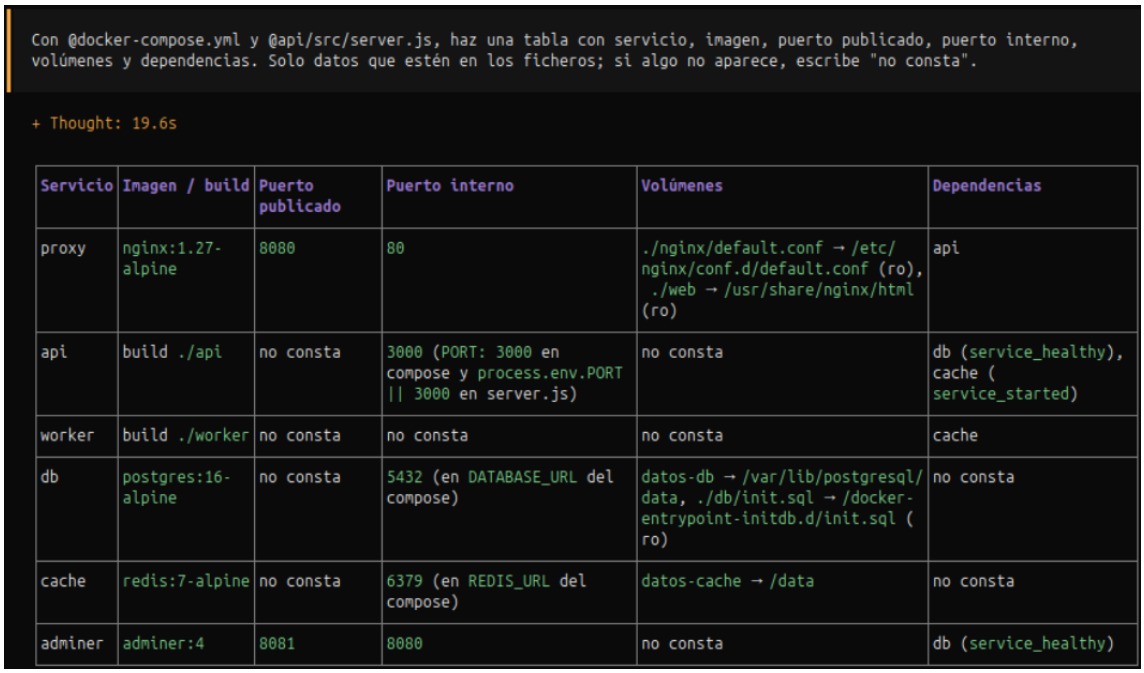
1. Leer catálogo — El navegador pide /api/libros → nginx lo reescribe a http://api:3000/libros → la API busca en Redis;
   si está en caché responde sin tocar la BD, si no consulta PostgreSQL y guarda el resultado 60 s en Redis.
2. Crear pedido — POST /pedidos con {libroId, unidades} → la API inserta en la tabla pedidos y hace LPUSH cola:facturas
   en Redis → responde 201 {pedido: id}.
3. Facturación asíncrona — El worker está bloqueado en BRPOP cola:facturas; al recibir el id, loguea "Generando la
   factura del pedido N" (por ahora no genera el archivo, solo simula el trabajo).

Decisiones de diseño clave

- Desacoplamiento mediante cola: la API no factura; emite un evento a Redis y el worker lo consume. Si el worker se cae,
  la cola retiene los pendientes (con el riesgo de perder hasta 60 s + lo esté en RAM si no hay AOF).
- Caché-aside: evita saturar PostgreSQL en el listado de libros, que es la lectura más frecuente.
- Puertos: solo nginx (8080) y adminer (8081) son públicos; API, BD y Redis viven en la red por defecto interna del
  Compose — el exterior no puede alcanzarlos.
- Orden de arranque: la API espera a que la BD pase el healthcheck (pg_isready) antes de arrancar; todo con restart:
  unless-stopped.
- Estado: los volúmenes datos-db y datos-cache persisten datos al recrear contenedores; init.sql solo se ejecuta en el
  primer arranque.

Puntos débiles de la arquitectura (relevantes para producción)

- jsonwebtoken está declarado pero no se usa: POST /pedidos está sin autenticar.
- /salud responde ok aunque la BD o Redis estén caídos.
- Adminer expuesto en 8081 sería un acceso directo a los datos.
- Sin TLS, sin migraciones de esquema (solo el init.sql de la primera ejecución) y sin lockfile de dependencias.
```



Al repetir el ejercicio 2 con dos prompts distintos, la diferencia más clara fue el formato y la fiabilidad. Con el prompt vago, el agente devolvió una explicación larga que mezclaba lo que hay en el proyecto con generalidades teóricas y citó pocos ficheros.

Con el prompt preciso, pedí una tabla y le indiqué que escribiera “no consta” cuando no hubiera dato, la respuesta fue una tabla con las columnas pedidas, basada en los ficheros. También marcó como “no consta” los puertos internos de db y cache, que no aparecen en el compose.

La diferencia se debe a que el agente rellena lo que no se le pide con lo que “suele ser” una arquitectura. Cuando el prompt le limita los ficheros, el formato y una regla contra la invención, tiene menos margen para inventar y la respuesta resulta más útil y precisa. Aun así, el prompt preciso no elimina la necesidad de comprobar. Incluso con él, hay que contrastar cada dato con los ficheros porque la evidencia es lo que se ve en el comando y no lo que afirma el agente.

Registro de prompts Mantén durante toda la práctica el fichero `prompts.md` con una tabla como esta (una fila por prompt importante):

Nº	Prompt (resumido)	Qué hizo el agente	Qué corregí yo
1	Explica qué hace este proyecto y qué ficheros lo componen	Resumen correcto; se inventó una carpeta tests/	Le indiqué que no existe y lo comprobé con ls

Entrega

Entrega por el aula virtual un PDF con las capturas y las respuestas de cada ejercicio (debajo de cada uno), el fichero `prompts.md` y el enlace al repositorio. Una captura de la respuesta del agente **sin la comprobación** correspondiente no puntúa.